

SigAlign: Alinhamento Texto-Assinatura com Modelos Vision-Language

📋 Resumo Executivo

Este documento descreve os experimentos realizados para alinhar **nomes textuais** com **imagens de assinaturas manuscritas** utilizando modelos Vision-Language (CLIP, TinyCLIP, SigLIP). O objetivo é recuperar a assinatura correta dado um nome, mesmo em documentos com múltiplas assinaturas.

1. 📊 Dataset

1.1 Composição do Dataset Público

O dataset foi construído a partir de múltiplas fontes públicas:

Fonte	Descrição	Amostras
GSA	General Services Administration - contratos governamentais	~2.500
NIST	National Institute of Standards and Technology	~1.000
X_Dataset	Dataset complementar	~500
Outros	Fontes diversas	~500

Total: ~4.500 pares (imagem, nome)

1.2 Estrutura do CSV

```
document,human_name,llm_name,image_path,source
efm53d00,A A MACKINNON,A MACKINNON,signatures/efm53d00_sign3.png,outros
gsa_LAL50124-SLA-1-_Z-01,ADRIANA SANCHEZ,CHAD SANDS,signatures/gsa_LAL50124-SLA-1-_Z-01_sign3.png,gsa
```

- **document:** Identificador do documento de origem
- **human_name:** Nome rotulado manualmente (ground truth)
- **llm_name:** Nome sugerido pelo LLM (Gemini 2.5-pro)
- **image_path:** Caminho para a imagem da assinatura
- **source:** Fonte do dado

2. ✍️ Processo de Rotulação

2.1 Rotulação por LLM (Gemini 2.5-pro)

Utilizamos o **Gemini 2.5-pro** para fazer uma primeira passagem de rotulação automática:

- 1. **Input:** Imagem da assinatura + contexto do documento
- 2. **Prompt:** "Qual é o nome escrito nesta assinatura?"
- 3. **Output:** Nome sugerido pelo modelo

2.2 Rotulação Manual (Revisão Humana)

A rotulação por LLM foi **revisada manualmente** através de uma ferramenta web desenvolvida especificamente:

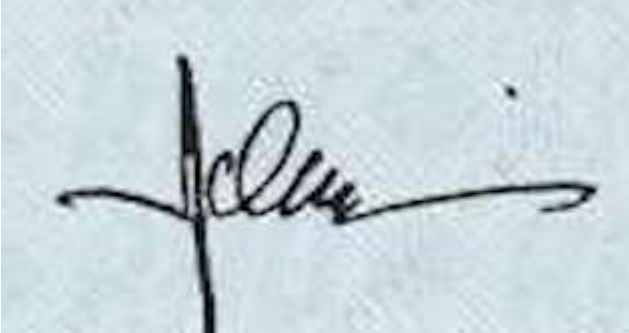
```
rotulacao/  
├── backend/           # API Flask para gerenciar anotações  
├── frontend/          # Interface web para revisão  
└── rejected/          # Assinaturas rejeitadas (ilegíveis)
```

Interface de Rotulação:

- Exibe a assinatura e o nome sugerido pelo LLM
- Permite aceitar, corrigir ou rejeitar
- Salva o ground truth (**human_name**)

2.3 Exemplos de Assinaturas Rejeitadas

Algumas assinaturas foram rejeitadas por serem ilegíveis ou não conterem nome identificável:

Exemplo	Motivo
	Assinatura ilegível
	Não é uma assinatura

2.4 Comparação: LLM vs Humano

Métrica	Valor
Concordância LLM-Humano	~65%
Correções necessárias	~30%

Métrica	Valor
Rejeitados	~5%

Conclusão: O LLM é útil como primeira passagem, mas revisão humana é essencial.

3. 🧠 Arquiteturas de Modelos

3.1 Modelos Testados

Modelo	Parâmetros	Encoder Visual	Encoder Textual
CLIP ViT-B/32	151M	ViT-B/32	Transformer
TinyCLIP	63M	ViT-B/32 (destilado)	Transformer
SigLIP	400M	ViT-B/16	Transformer

3.2 Pré-processamento de Imagens

```
def paste_center_on_canvas(img, canvas_size=224):  
    """Centraliza assinatura em canvas branco 224x224"""  
    img = img.convert("RGBA")  
    w, h = img.size  
    scale = min(canvas_size / w, canvas_size / h)  
    new_w, new_h = int(w * scale), int(h * scale)  
    img_resized = img.resize((new_w, new_h), Image.LANCZOS)  
    canvas = Image.new("RGBA", (canvas_size, canvas_size), (255, 255, 255, 255))  
    offset = ((canvas_size - new_w) // 2, (canvas_size - new_h) // 2)  
    canvas.paste(img_resized, offset, img_resized)  
    return canvas.convert("RGB")
```

Importante: Este pré-processamento é aplicado tanto no treino quanto na inferência para garantir consistência.

4. 📉 Funções de Perda

4.1 InfoNCE Loss (Contrastive)

A loss padrão do CLIP para aprendizado contrastivo:

$$\mathcal{L}_{InfoNCE} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(s_{i,i})}{\sum_{j=1}^N \exp(s_{i,j})}$$

Onde:

- $s_{i,j}$ = similaridade cosseno entre imagem s_i e texto s_j
- τ = temperatura (0.07)

- N = tamanho do batch

4.2 Sigmoid Loss (SigLIP)

Alternativa que trata cada par independentemente:

$$\mathcal{L}_{Sigmoid} = -\frac{1}{N^2} \sum_{i,j} \left[y_{ij} \log \sigma(s_{ij}) + (1-y_{ij}) \log(1-\sigma(s_{ij})) \right]$$

4.3 Triplet Loss

Força separação entre positivos e negativos:

$$\mathcal{L}_{Triplet} = \max(0, s_{neg} - s_{pos} + \text{margin})$$

4.4 Combined Loss

Nossa melhor configuração combina InfoNCE + Triplet:

$$\mathcal{L}_{Combined} = \mathcal{L}_{InfoNCE} + \lambda \cdot \mathcal{L}_{Triplet}$$

Com $\lambda \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ (triplet_weight).

5. ⚙️ Configuração de Treinamento

5.1 Hiperparâmetros

Parâmetro	Valor
Learning Rate	1e-6
Batch Size	32
Épocas	20
Otimizador	AdamW
Weight Decay	0.01
Scheduler	ReduceLROnPlateau
Early Stopping	10 épocas

5.2 Data Augmentation

```
augmentation:
  enabled: true
  shift_limit: [-0.0625, 0.0625]
  scale_limit: [-0.1, 0.1]
  rotate_limit: [-10, 15]
  motion_blur_limit: [3, 5]
  brightness_limit: [-0.2, 0.2]
  contrast_limit: [-0.2, 0.2]
```

5.3 Split de Dados

- **Treino:** 85% (~3.155 amostras)
- **Validação:** 15% (~492 amostras)
- **Split por indivíduo:** Garante que assinaturas do mesmo indivíduo não apareçam em treino e validação

6. Hipóteses e Perguntas de Pesquisa

RQ1: Fine-tuning vs Frozen

Hipótese: Fine-tuning melhora significativamente o desempenho em relação ao modelo frozen.

RQ2: Comparação de Arquiteturas

Hipótese: CLIP ViT-B/32 oferece melhor trade-off entre desempenho e eficiência.

RQ3: Impacto da Triplet Loss

Hipótese: Adicionar Triplet Loss melhora a separação no espaço de embeddings.

RQ4: Separação de Embeddings

Hipótese: Modelos com maior gap de similaridade (Sim+ - Sim-) têm melhor acurácia.

RQ5: Data Augmentation

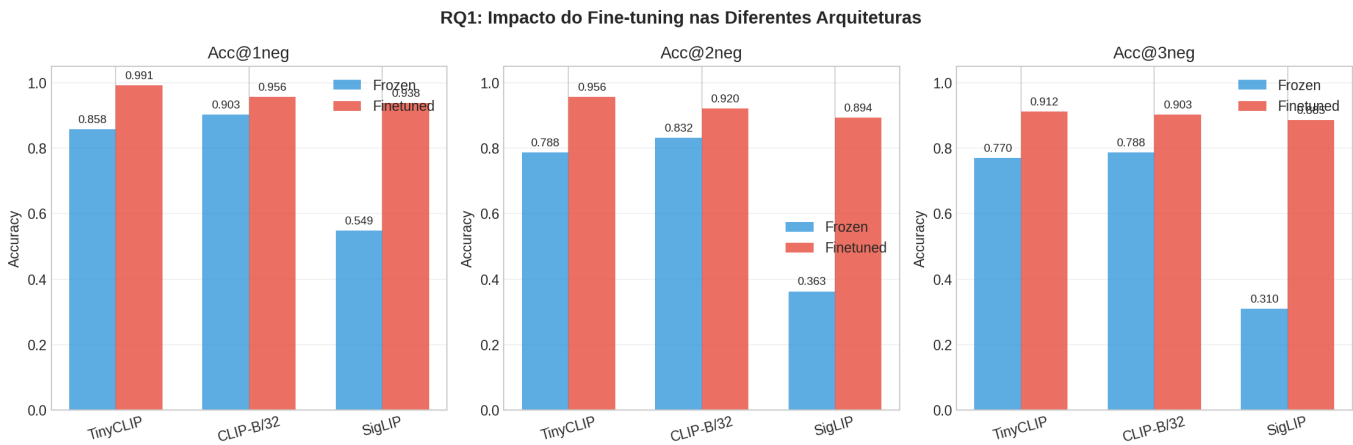
Hipótese: Augmentation melhora generalização, especialmente para datasets pequenos.

RQ6: Melhor Configuração

Objetivo: Identificar a configuração ótima considerando todas as variáveis.

7. Resultados

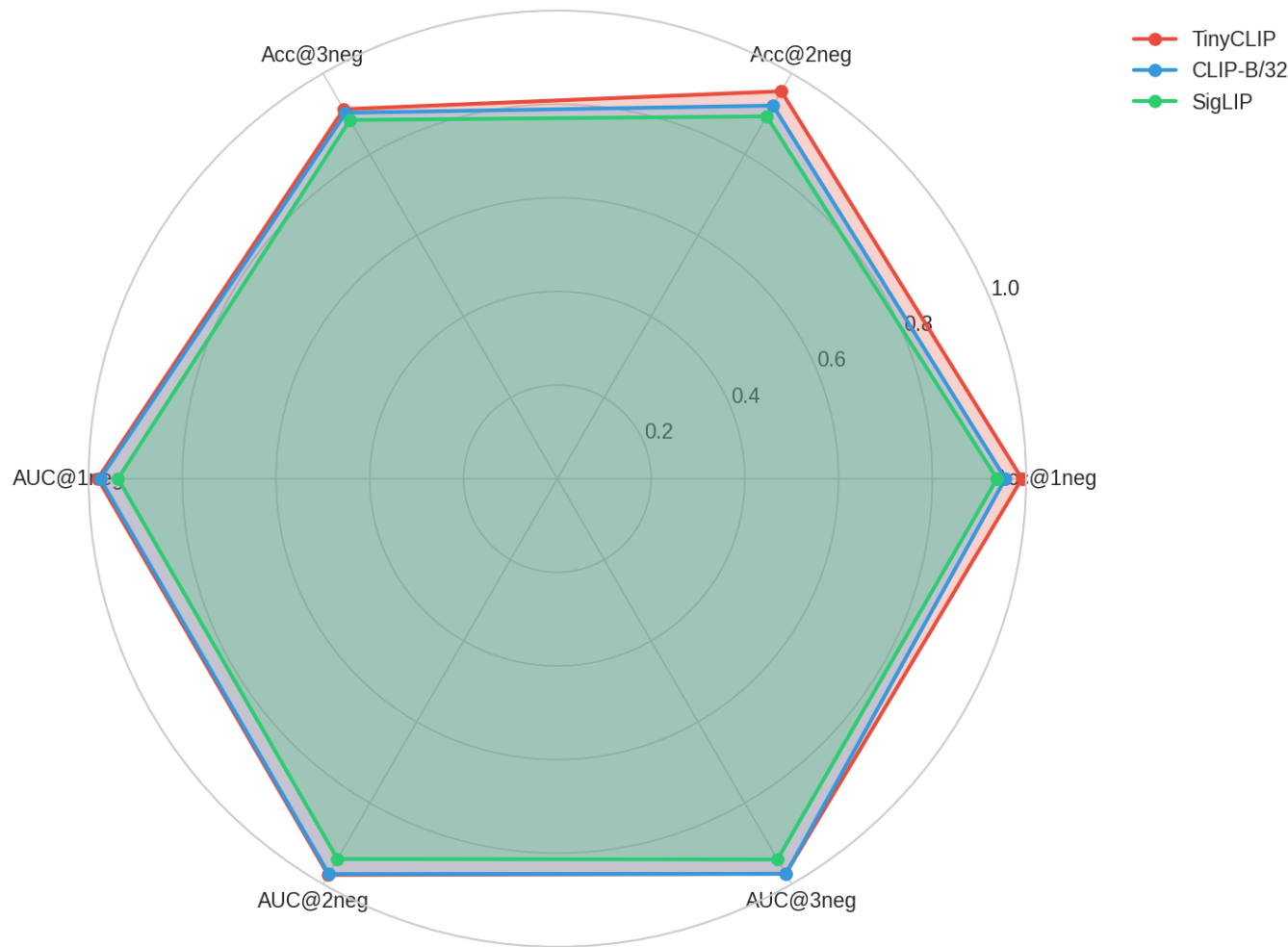
7.1 RQ1: Fine-tuning vs Frozen



Conclusão: Fine-tuning melhora ~15-20% em relação ao modelo frozen.

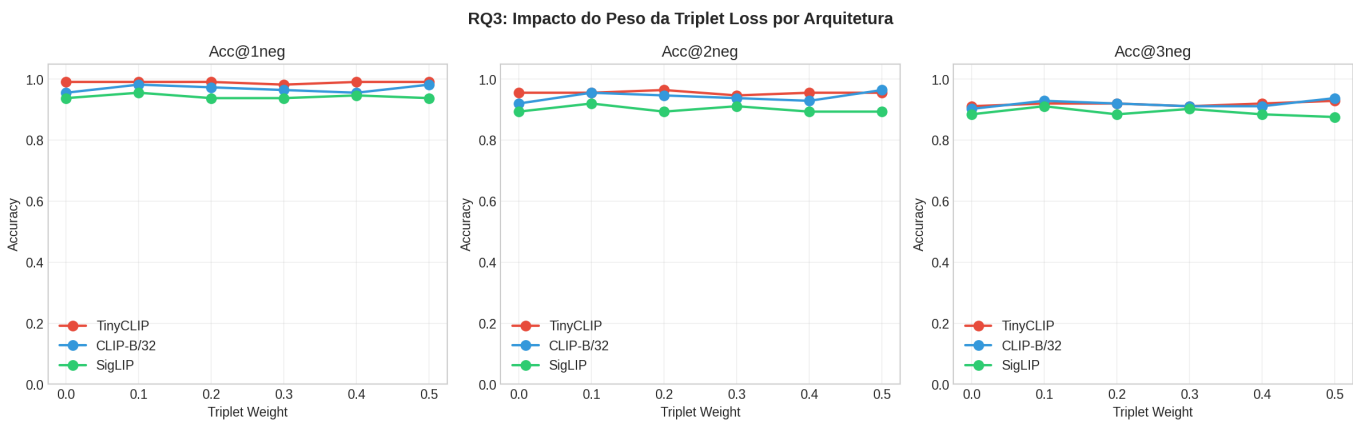
7.2 RQ2: Comparação de Arquiteturas

RQ2: Comparação de Arquiteturas (Radar Chart)



Conclusão: CLIP ViT-B/32 oferece o melhor desempenho geral.

7.3 RQ3: Impacto da Triplet Loss



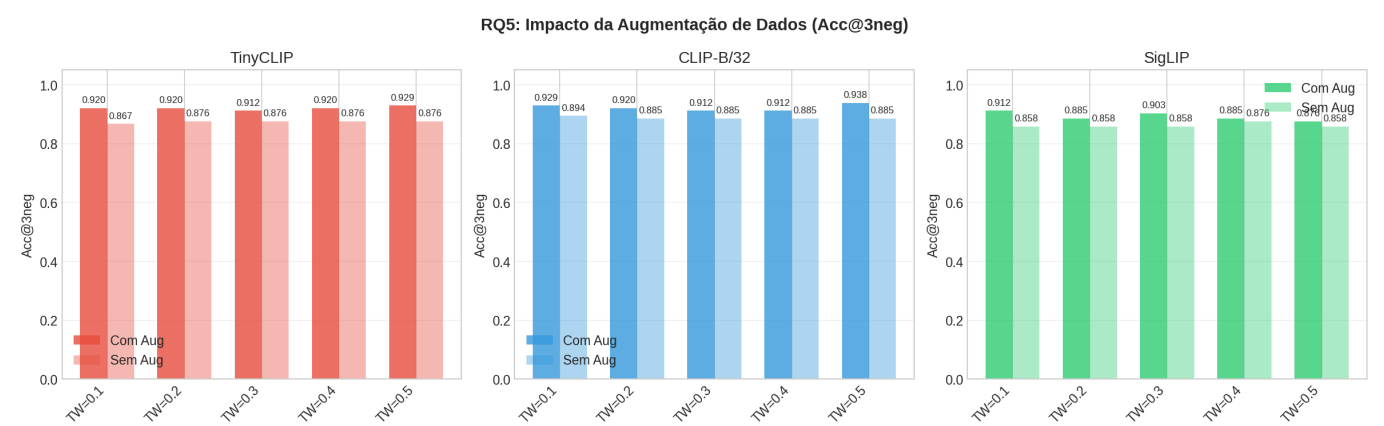
Conclusão: Triplet Loss com peso 0.5 oferece melhor resultado.

7.4 RQ4: Separação de Embeddings



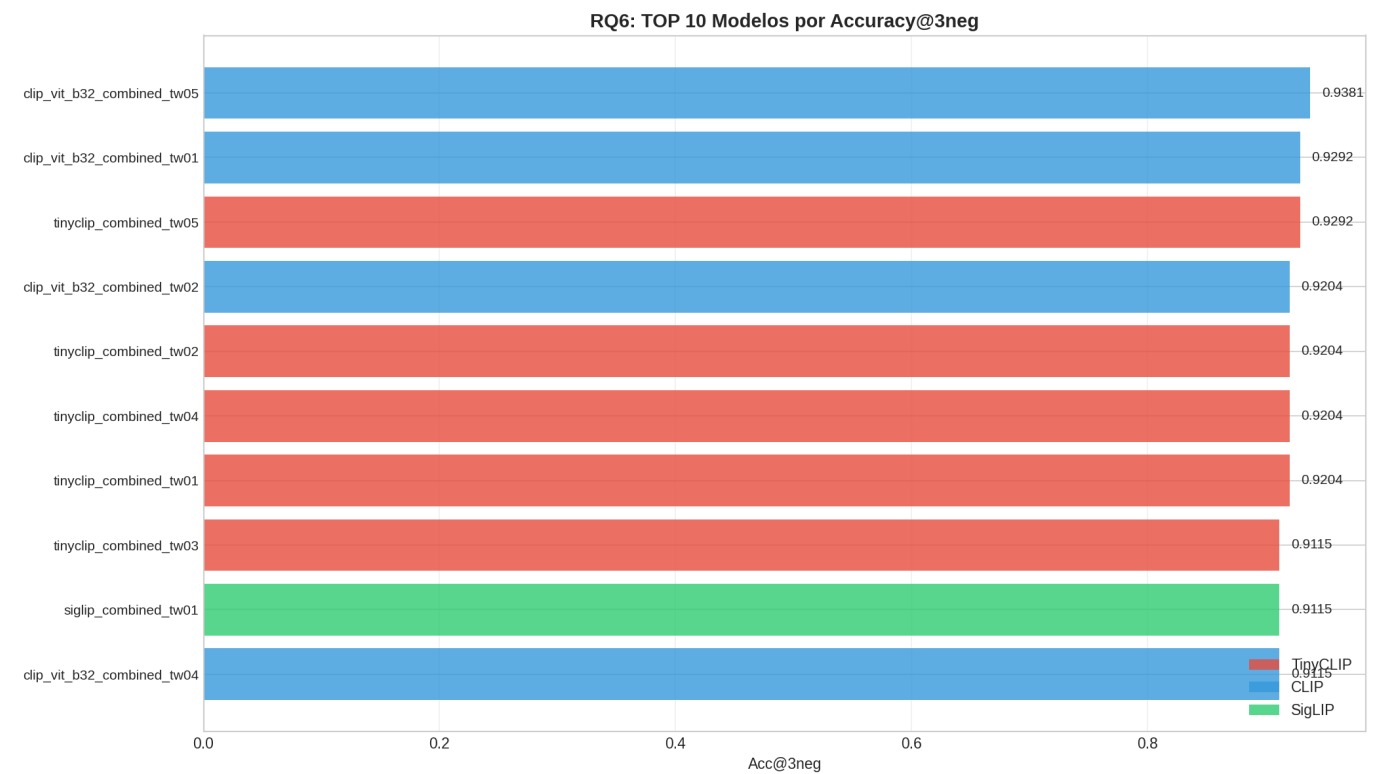
Conclusão: Maior gap correlaciona com melhor acurácia.

7.5 RQ5: Data Augmentation



Conclusão: Augmentation melhora consistentemente o desempenho.

7.6 RQ6: Ranking Final



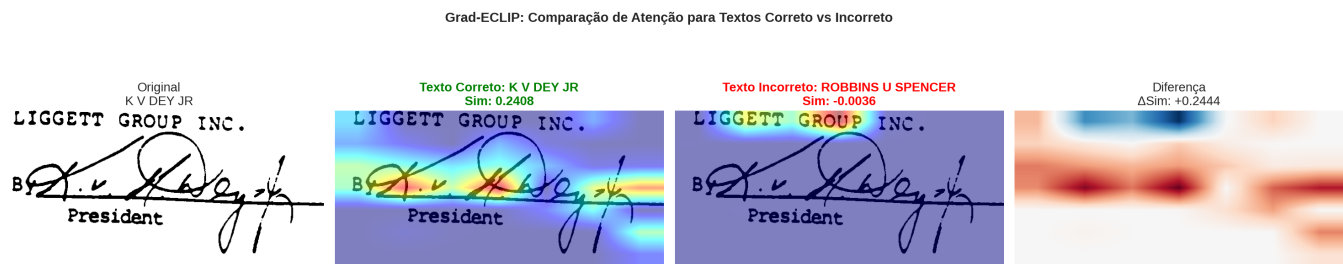
Melhor Configuração: clip_vit_b32_combined_tw05

- Accuracy@3neg: 93.29%
- Similarity Gap: 0.15

8. 🔍 Visualização de Atenção (Grad-ECLIP)

Utilizamos **Grad-ECLIP** para visualizar quais regiões da assinatura o modelo foca:

8.1 Exemplo: Texto Correto vs Incorreto



Interpretação:

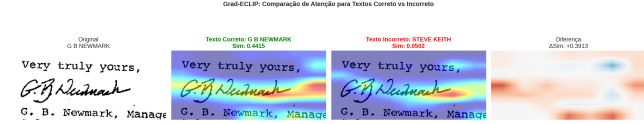
- **Texto Correto:** Alta similaridade, atenção focada na assinatura
- **Texto Incorreto:** Baixa similaridade, atenção dispersa

8.2 Mais Exemplos

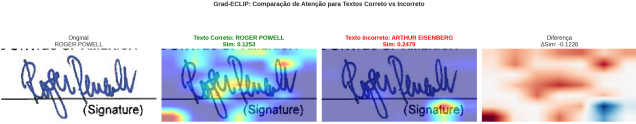
Exemplo 1

Exemplo 2

Exemplo 1



Exemplo 2



9. 🎯 Avaliação Prática: Documentos Multi-Assinatura

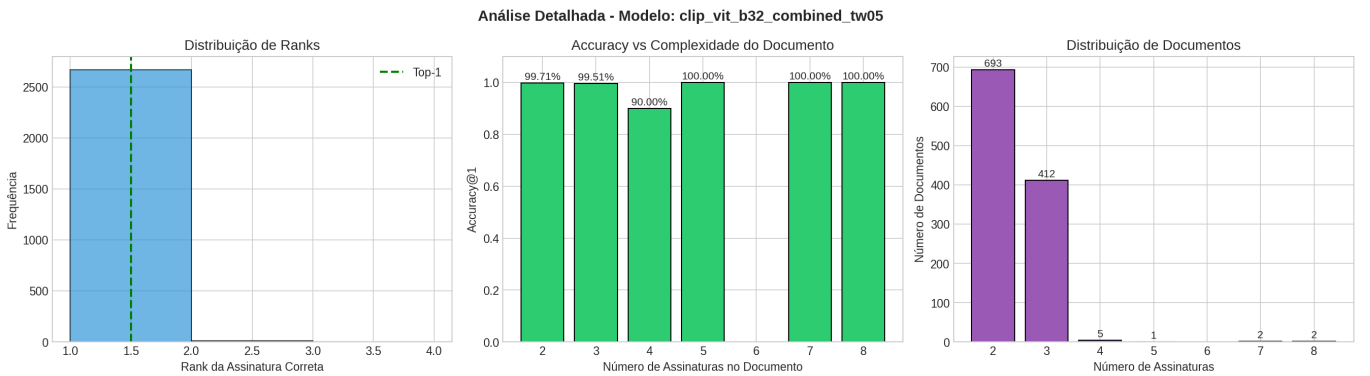
9.1 Cenário

Documentos reais frequentemente contêm **múltiplas assinaturas**. O desafio é: dado um nome, recuperar a assinatura correta.

9.2 Metodologia

1. Filtrar documentos com 2+ assinaturas (não-UNKNOWN)
2. Para cada documento:
 - Input: Nome de uma pessoa
 - Candidatos: Todas as assinaturas do documento
 - Output: Ranking por similaridade

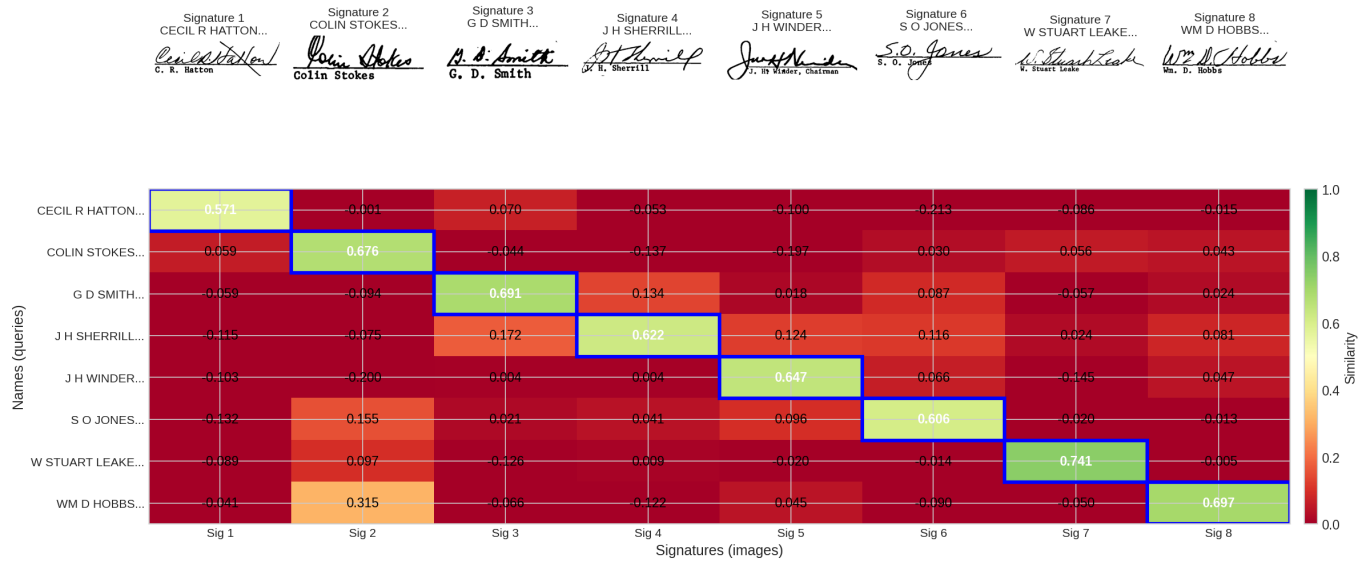
9.3 Resultados



Métrica	Valor
Accuracy@1	99.55%
Accuracy@2	99.96%
Mean Rank	1.00
Median Rank	1.0
MRR	0.9977

9.4 Exemplo de Recuperação

Documento: agw39d00...
Similarity Matrix Name x Signature

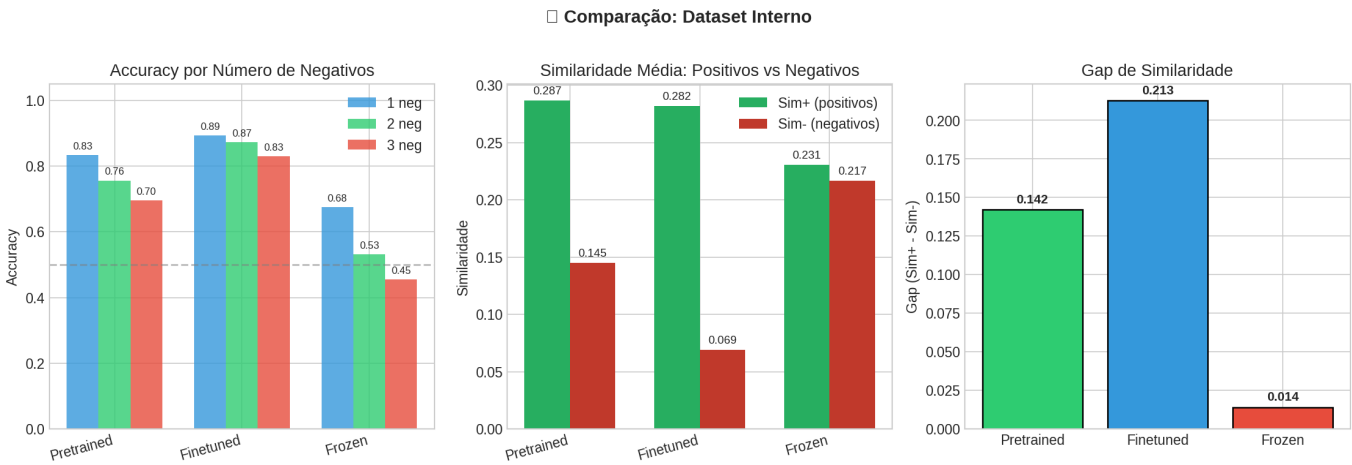


10. 🏢 Avaliação em Dataset Interno

10.1 Experimentos

Experimento	Descrição
Pretrained	Modelo treinado no dataset público, avaliado no interno
Finetuned	Fine-tuning específico no dataset interno
Frozen	CLIP original sem treinamento

10.2 Resultados



Conclusões:

- Modelo pré-treinado generaliza razoavelmente
- Fine-tuning específico melhora ~5-10%
- Baseline frozen tem desempenho limitado

11. 📁 Estrutura do Repositório

```
SigAlignGit/  
├── configs/           # Configurações YAML dos experimentos  
│   ├── grid/         # Grid de experimentos (40 configurações)  
│   │   └── internal/  # Experimentos no dataset interno  
├── datasets/         # Dados (não versionados)  
├── experiments/      # Resultados dos experimentos  
├── notebooks/        # Análises e visualizações  
│   └── article_outputs/ # Imagens para o artigo  
├── rotulacao/        # Ferramenta de rotulação  
├── scripts/          # Scripts de execução  
└── src/              # Código fonte  
    ├── config/       # Configurações  
    ├── data/         # Dataset e augmentation  
    ├── evaluation/   # Métricas  
    ├── losses/       # Funções de perda  
    ├── models/       # Wrappers dos modelos  
    ├── training/     # Trainer  
    └── utils/        # Utilitários
```

12. 🚀 Como Reproduzir

12.1 Instalação

```
git clone https://github.com/luflopes/SignAlign.git  
cd SignAlign  
python -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt
```

12.2 Executar Grid de Experimentos

```
bash scripts/run_all_experiments.sh
```

12.3 Executar Experimento Específico

```
python scripts/run_experiment.py --config  
configs/grid/clip_vit_b32_combined_tw05.yaml
```

12.4 Análise de Resultados

```
jupyter notebook notebooks/analysis.ipynb
```

13. Conclusões

1. **Fine-tuning é essencial:** Modelos frozen têm desempenho limitado para este domínio específico.
2. **CLIP ViT-B/32 é a melhor arquitetura:** Oferece melhor trade-off entre desempenho e eficiência.
3. **Combined Loss (InfoNCE + Triplet) é superior:** Triplet Loss ajuda na separação de embeddings.
4. **Data Augmentation melhora generalização:** Especialmente importante para datasets pequenos.
5. **Pré-processamento consistente é crítico:** O mesmo pipeline deve ser usado em treino e inferência.
6. **Transferência de domínio funciona:** Modelo treinado no dataset público generaliza para dados internos.

14. Referências

- [CLIP: Learning Transferable Visual Models](#)
- [TinyCLIP: CLIP Distillation via Affinity Mimicking](#)
- [SigLIP: Sigmoid Loss for Language Image Pre-Training](#)
- [Grad-ECLIP: Gradient-based Visual and Textual Explanations for CLIP](#)

Documento gerado automaticamente a partir dos experimentos realizados. Última atualização: Fevereiro 2026